Core Program Logic for Vibe Coding

Module One

Agenda

- Variables
- Data Structures
- Pandas

Why Core Logic Still Matters in Vibe Coding

Better Prompts

Precise technical vocabulary ("early return when null", "map over the list") produces far more accurate AI output.

Critical Review

You can read the generated code and judge whether the logic is correct, even if you never type the syntax yourself.

Faster Debugging

When the AI gets stuck, you can describe the failure mode accurately instead of only pasting error messages.

Program Logic for vibe coding (Module one)



Overview of Python



Setup and install tools (Anaconda)



Data types



Data objects



Overview of data processing with pandas (filtering)

Overview of Python

Why we use Python

- Open source
- **Easy to use**
- Top 2 popular programming languages in the world
- Lots of users and developers with big user community
- **Lots of useful packages**
  - Numpy
  - Pandas
  - Matplotlib
  - Tensorflow
  - Beatifulsoup
  - Selenium
  - Scikit learn
  - And more…
- Most of the popular brokers have Python APIs nowadays
- Good support on data analytics tasks (use of different ML algorithms)

Typical use cases of Python

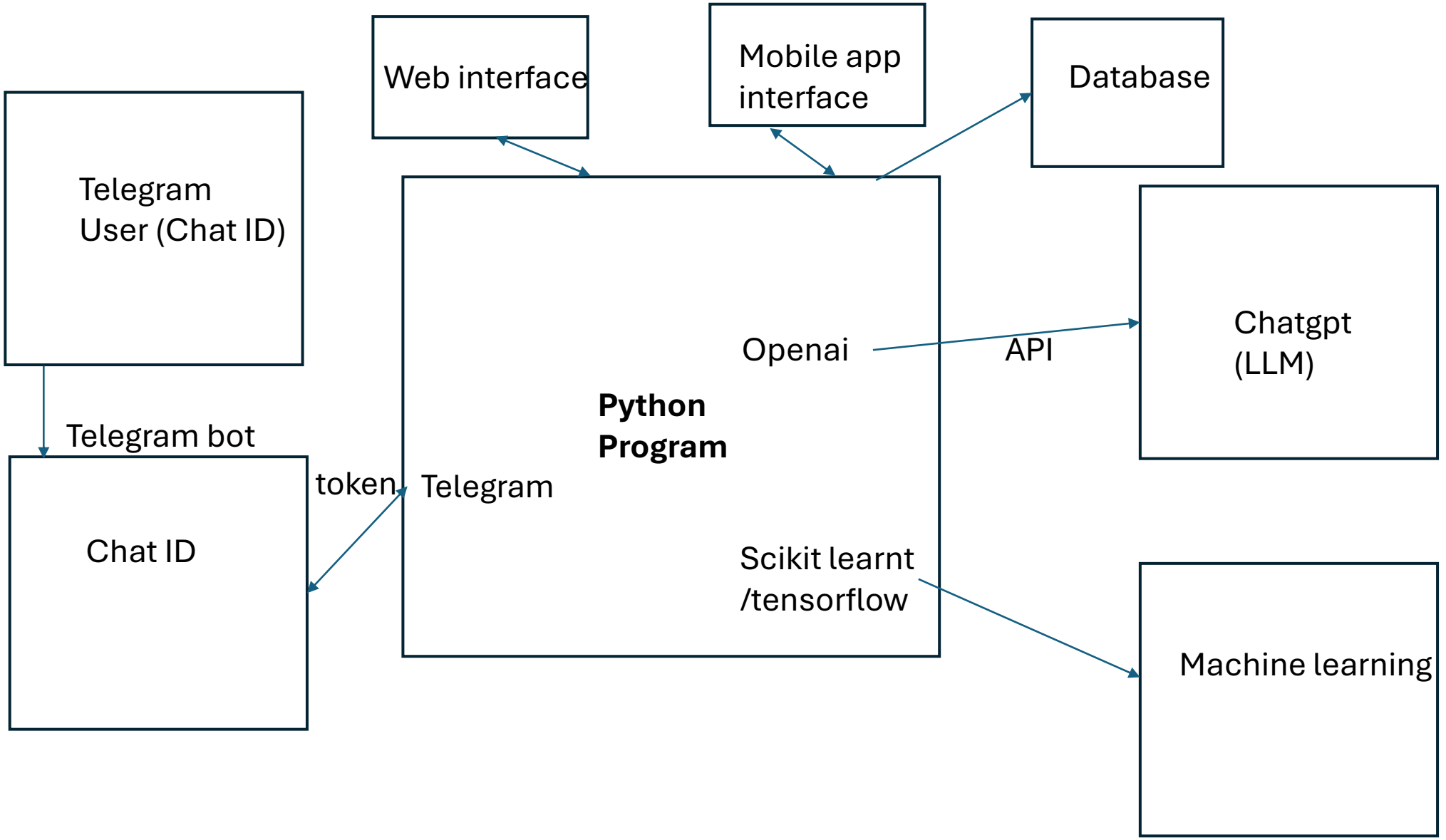


DATA ANALYTICS/DATA ANALYSIS



TASK AUTOMATION

Use of Python with Different Applications



Setup of IDE

Python

Download Python

- https://www.python.org/downloads/

- IDE for Python
- Default: IDLE
- Others: Pycharm
  https://www.jetbrains.com/pycharm/download/#section=windows
- Jupyter notebook

  https://www.anaconda.com/download/

We will use this one during the class !

Use of Jupyter Notebook Data Analysis

- Data Scientist normally uses
- Jupyter notebook for data analysis
  - Statistical analysis
  - Data visualization

Example of using decision tree and random forest

Import data

[1]:
```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree
from plotnine import *
%matplotlib inline
```

```
!pip install pandas plotnine
```

[2]:
```
loc ="http://archive.ics.uci.edu/ml/machine-learning-databases/"
ds  ="breast-cancer-wisconsin/breast-cancer-wisconsin.data"
url =loc+ds
data=pd.read_csv(url,sep=",",header=None)
```

[3]:
```
data.columns =["ID", "clumpThickness", "sizeUniformity",
               "shapeUniformity", "maginalAdhesion",
               "singleEpithelialCellSize", "bareNuclei",
               "blandChromatin", "normalNucleoli", "mitosis", "class"]
```

[4]:
```
data.head()
```

[4]:

	ID	clumpThickness	sizeUniformity	shapeUniformity	maginalAdhesion	singleEpithelialCellSize	bareNuclei	blandChromatin	normalNucleoli	mitosis	class
0	1000025	5	1	1	1	2	1	3	1	1	2
1	1002945	5	4	4	5	7	10	3	2	1	2
2	1015425	3	1	1	1	2	2	3	1	1	2
3	1016277	6	8	8	1	3	4	3	7	1	2
4	1017023	4	1	1	3	2	1	3	1	1	2

Task Execution/Automation

While jupyter notebook is used for data analysis, python script will be used for task execution and automation.

The script will be executed with a command line under command prompt such as

>python abc.py

Which means it will use python to execute the python script line by line and from top to bottom.

Sample python script (.py)

```
import pandas as pd
import numpy as np
import yfinance as yf
import time
#import telegram
# from tabulate import tabulate
import matplotlib.pyplot as plt
from datetime import date
import asyncio
#import talib as TA


path="C:/temp/"

# Token and chat ID used in telegram
TOKEN=""
chat_id=""

# Starting date which is used to generate the chart
date1="2021-09-01"
today=date.today().strftime("%d%m%Y")

# Import excel with stock codes and create a list L_stock which stores the
# stock codes

L_stocks=pd.read_excel(path+"Stock_monitoring_list2.xlsx")["Stock code"].tolist()
L_stocks.append("^HSI")
L_stocks_names=pd.read_excel(path+"Stock_monitoring_list2.xlsx")["Name"].tolist()
L_stocks_names.append("HSI")

# To download the stock data one by one and save it as a dictionary
def getdata(L):
    D1={}
    for i in L:
        df=yf.download(i,"2021-01-01",progress=False)
        message="Now downloading data:"+i
        print(message)
        D1[i]=df
        time.sleep(2)
    return(D1)

# Based on the dictionary above generates the result with the restuen of previous 5 days and
# previous 20 days

def table_gen1(D):
    L_return_1day=[]
    L_return_5days=[]
    L_return_20days=[]
    for i in L_stocks:
        return1day=round(100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-2])/D[i]["Close"].iloc[-2],1)
        return5days=round(100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-5])/D[i]["Close"].iloc[-5],1)
        return20days=round(100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-20])/D[i]["Close"].iloc[-20],1)
        L_return_1day.append(return1day)
        L_return_5days.append(return5days)
```

Python 101

What we are going to cover

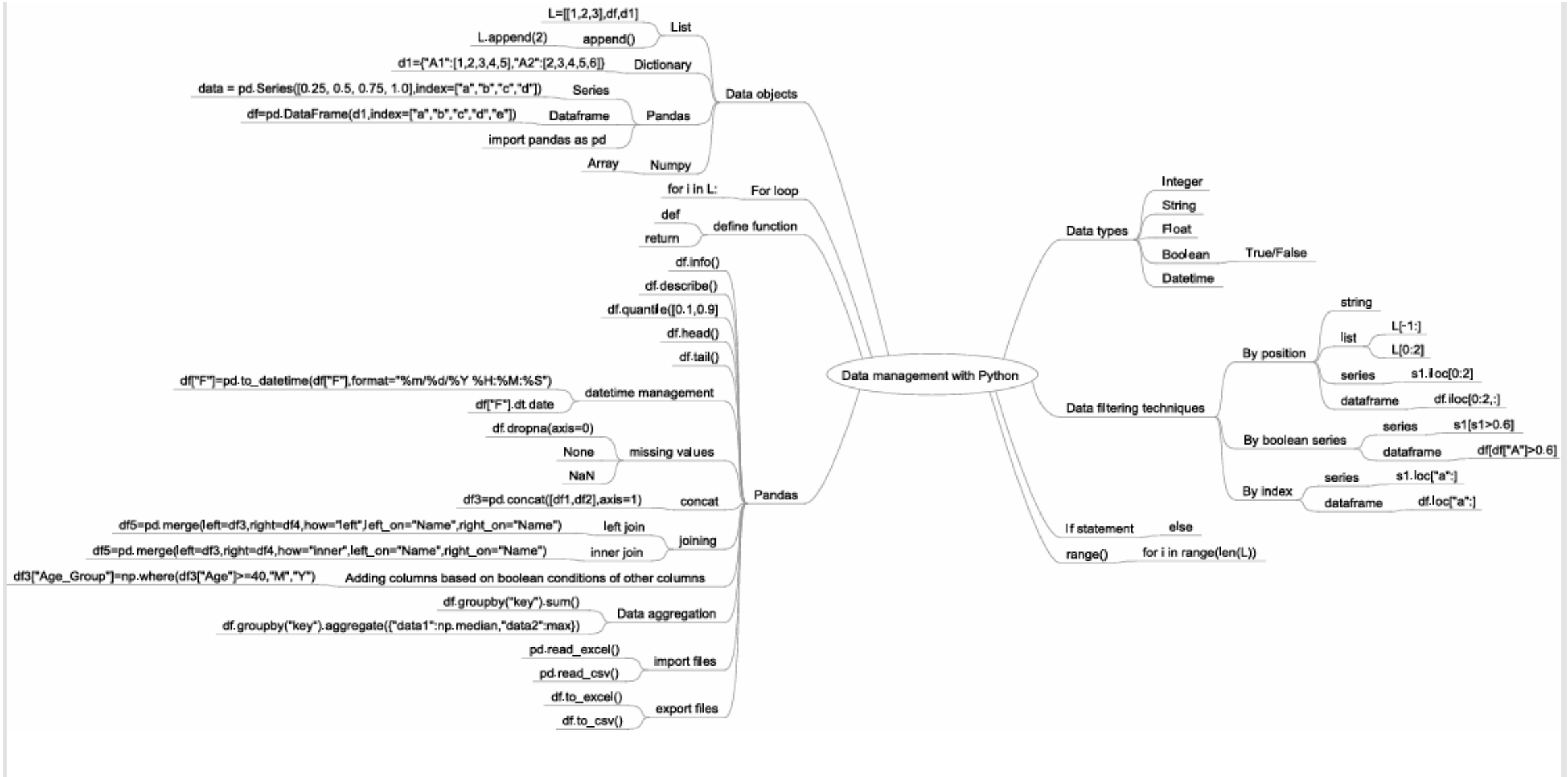


Table of Contents

01 **Python Foundations & Core Logic**

Print · Data Types · Variables · Lists · Dictionaries

02 **Working with Real Data (DataFrames)**

yfinance · Columns · Calculated Logic · Statistics · Export

03 **Filtering Logic in Pandas**

iloc · Boolean Masks · loc · shift() · Daily Returns

04 **Aggregation & Grouping Logic**

groupby · Aggregate · Multi-level · Range Metrics

05 **Advanced Data Processing Logic**

Datetime · Missing Values · Concat · Conditions · Joins

Data types in Python

- booleans
- integers
- floats
- Strings
- Datetime

- Use type() function to check data type
- Use int()/float()/str()/bool() for the conversion of data types

Data Objects

- List
- Dictionary
- Series
- Dataframe

List



- One dimensional (1D) data object
- Each data object within the list is independent and could have different data types
- You could treat it as a 1-D buffer for data processing
- Identify each of the list item by position
- Python position starts with Zero numbering

Dictionary



- One dimensional (1D) data object
- Each data object within the dictionary is independent and could have different data types
- Each data object will have the key assigned to it.
- Hence we also call a dictionary as key-values pairs

Series

Data object A	Data object B	Data object C
101	**102**	**103**



- One dimensional (1D) data object which are provided by Pandas
- Each data object within the series will have the same data type
- The series has its index to perform efficient data fetching

Dataframe

	A	B	C
0	Peter	1000	True
1	Susan	2000	True
2	David	4000	False
3	Alan	6000	False
4	Nancy	5000	True
5	Alice	70000	False

Index

- It is a 2-D data object which comes from pandas
- Each column has the same type
- Each row has index for efficient data fetching

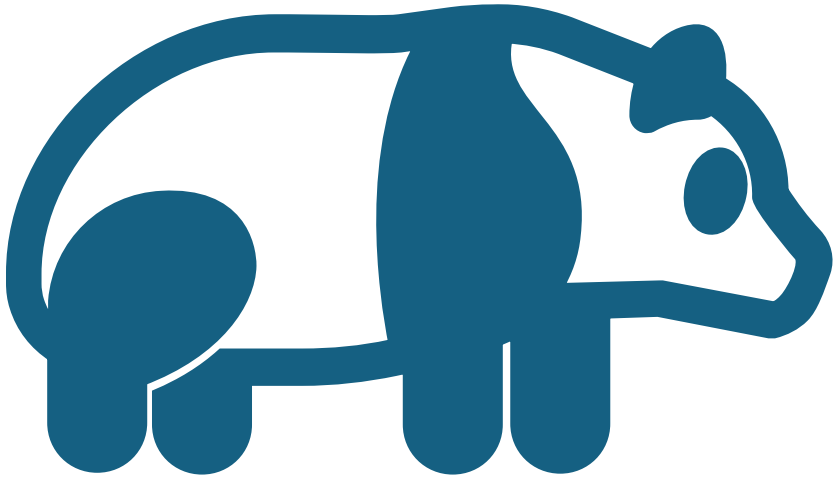
Sample Code Snippets

Refer to separate notebook for details

- Installing package
- Activate package
- Overview of data management task with pandas
  - Add column
  - Perform statistical distribution analysis
  - Import/Export Excel file
  - Simple data visualization (e.g. line chart)

Working with Real Data (DataFrames)

01	**Loading External Data**	Using yfinance to download real market data (^HSI) into a DataFrame.
02	**DataFrame Structure**	Understanding rows, columns, index, and the shape of tabular data.
03	**Selecting Columns**	Accessing series from a DataFrame and working with individual columns.
04	**Creating New Columns**	Calculated logic: High–Open difference and High–Low percentage range.
05	**Basic Statistical Logic**	describe(), mean(), max(), min(), quantile() — summarising distributions.
06	**Exporting Results**	Persisting analysis results to Excel for sharing and further work.



Data Filtering with Pandas

Filtering methods in Pandas

- By position (with iloc[])
- By Boolean series
- By index (with loc[])

Filtering methods of Pandas (By position)

```
In [7]: import pandas as pd
        data = pd.Series([0.25,0.5,0.75,1.0],index=['a','b','c','d'])
        data
```

```
Out[7]: a    0.25
        b    0.50
        c    0.75
        d    1.00
        dtype: float64
```

Starting index

Ending index

```
In [8]: data.iloc[0:2]
```

```
Out[8]: a    0.25
        b    0.50
        dtype: float64
```

Filtering methods of Pandas (By Boolean series)

```
data
```

```
a    0.25
b    0.50
c    0.75
d    1.00
dtype: float64
```

\> 0.6

```
a 0.6
b 0.6
c 0.6
d 0.6
```



```
data>0.6
```

```
a    False
b    False
c     True
d     True
dtype: bool
```

```
data[data>0.6]
```

```
c    0.75
d    1.00
dtype: float64
```

Boolean series object

Filtering methods of Pandas (By index)

```
data.loc[["a","c"]]
```

```
a    0.25
c    0.75
dtype: float64
```

To filter and extract the items with index=“a” and “c”

```
data.loc["b":]
```

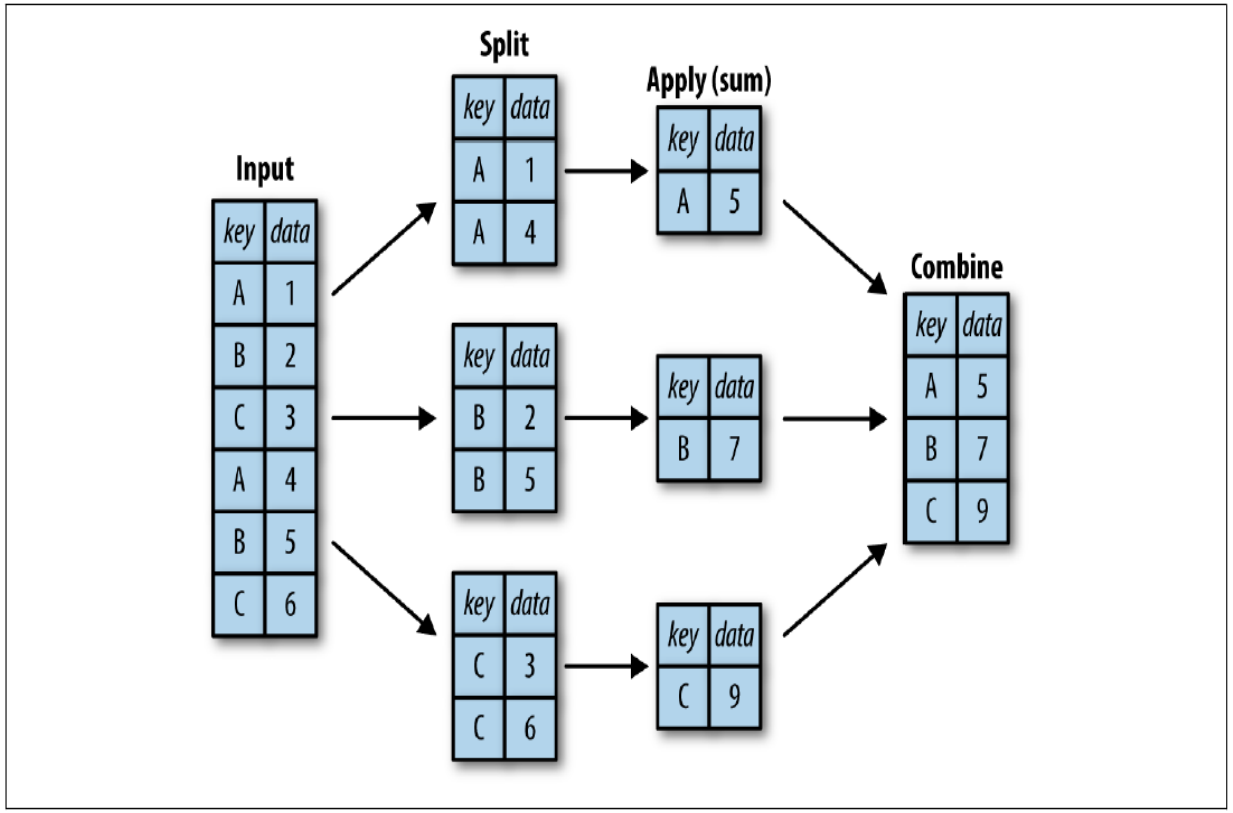
```
b    0.50
c    0.75
d    1.00
dtype: float64
```

To filter items with index values starting from “b”

Data Aggregation in Pandas

GroupBy: Split, Apply, Combine

- The split step involves breaking up and grouping a DataFrame depending on the value of the specified key.
- The apply step involves computing some function, usually an aggregate, transformation, or filtering, within the individual groups.
- The combine step merges the results of these operations into an output array.



Aggregation & Grouping Logic

Grouping by Categories

groupby() to split data into Year / Month buckets and process each group independently.

Single Aggregations

max(), min(), mean() on High / Low columns to summarise price extremes per period.

Multi-level Grouping

groupby(["Year", "Month"]) for hierarchical time-based analysis of market ranges.

Derived Metrics

Create Range = High − Low after aggregation to measure monthly volatility.

Multiple Aggregations

aggregate({"Range": ["mean", "max", "min", "std"]}) for full statistical profile by month.

Advanced Data Processing Logic

01

Datetime Management

pd.to_datetime(), extracting .dt.date / .dt.hour / .dt.minute from timestamps.

02

Handling Missing Values

Detect and remove nulls with dropna(). Understand axis and inplace behaviour.

03

Combining DataFrames

Concatenate multiple frames vertically or horizontally for unified datasets.

04

Conditional Column Creation

Add new columns based on filtering conditions of existing columns (logic branching).

05

Joining DataFrames

Left join, Inner join, multi-key joins with pd.merge() — relational thinking in Pandas.